

Atividade 4 - Módulo 4: LoRA e PEFT

Disciplina: Processamento de Dados e Fine-Tuning de Modelos · Prof. Ahirton Lopes, Ph.D.

Missão Prática #04

Esta atividade fecha o Módulo 4 inteiro: vocês vão aplicar, ao próprio contexto de trabalho, o processo completo de fine-tuning local via LoRA, conversão de dataset, validação de hiperparâmetros, comparação de posto, e decisão custo-benefício contra full fine-tuning, os instrumentos construídos ao longo dos quatro vídeos deste módulo.

Tempo e custo esperados

Treino local via LoRA (MLX ou equivalente) leva minutos, não horas, e não tem custo de nuvem nenhum se rodar na própria máquina. Se optarem por rodar via API gerenciada em vez de local, esperem tempo e custo parecidos com os da Missão Prática #03 (fine-tuning supervisionado): entre 13 minutos e cerca de 1 hora e 40 minutos de espera, poucos reais de custo.

Quem quiser ver como fica a chamada real pra API gerenciada antes de decidir: `lora-managed-api-preview-tool.js` (e o par em Python, `lora_managed_api_preview_tool.py`) monta a requisição HTTP real pra Together AI com a mesma configuração de LoRA do treino local, sem custo (modo preview, sem enviar nada, a menos que `TOGETHER_API_KEY` esteja configurada).

Passo 1 - Escolham um caso pequeno, já aprovado

Peguem um caso real do próprio trabalho de vocês que já passe no framework de quatro perguntas do Módulo 1.3 (aprovado, fine-tuning vale a pena), mas com volume mensal pequeno demais pra justificar o custo fixo de GPU alugada na nuvem, a mesma lógica financeira do Módulo 4.1. Rodem o NPV do Módulo 1.3 pra esse caso, comparando custo de GPU alugada (o valor de referência desta disciplina, R\$ 2.400 por job) contra custo de treinamento local, perto de zero, porque o hardware já existe.

Companion pronto pro Módulo 4.1: `regional-lora-vs-cloud-npv.js` (e o par em Python, `regional_lora_vs_cloud_npv.py`) já roda exatamente essa comparação de NPV, reaproveitando `calculateNPV` do `decision-framework-tool.js` do Módulo 1.3 sem duplicar lógica, contra um caso regional de referência de quatrocentos documentos por mês. Referência de como fica o script antes de adaptar pro caso de vocês.

Passo 2 - Treinem localmente, comparando pelo menos dois postos

Convertam seu dataset pro formato messages (role e content) que frameworks de treino local como o MLX-LM esperam. Rodem, ou documentem por que não foi possível rodar, um treino LoRA local contra esse dataset, comparando pelo menos dois postos diferentes, por exemplo quatro e oito, registrando parâmetros treináveis, val loss final, pico de memória e tamanho do checkpoint pra cada um, a mesma disciplina do Módulo 4.3.

Para quem não tem Mac com Apple Silicon e quer rodar de verdade mesmo assim, não só documentar que não deu: `colab-lora-training-notebook.ipynb` faz o mesmo treino LoRA, mesmo modelo, mesmo dataset, mesmo posto, via Hugging Face, rodável de graça numa GPU do Google Colab. Resultado real de referência: loss de validação caindo pra zero vírgula oito três zero cinco, mesma conclusão qualitativa do MLX. Opcional, vale a partir desta missão.

Passo 3 - Testem contra um exemplo difícil de propósito

Construam à mão pelo menos um exemplo de teste que estresse a tarefa de propósito: informação distratora, formato nunca visto no treino, ou qualquer outra armadilha real que force o modelo a entender a tarefa, não decorar posição de texto. Testem os adaptadores treinados contra esse exemplo, e documentem se o comportamento diverge entre os postos.

Antes de testar com o exemplo de vocês: `adapter-comparison-tool.js` (e o par em Python, `adapter_comparison_tool.py`) roda essa mesma comparação, com e sem adaptador, contra o exemplo real do Módulo 4.2 (recibo médico, Felipe Alves Monteiro), sem adaptador oitenta tokens batendo no limite sem chegar a um JSON, com adaptador vinte e oito tokens, JSON exato batendo com o gabarito. Referência de como fica o script antes de adaptar pro caso de vocês.

Companion parecido pro Módulo 4.3: `rank-adapter-comparison-tool.js` roda o mesmo exemplo difícil contra os três postos, rank 4, 8 e 16, todos acertam igual nesse caso, o mesmo resultado que o TP narra.

Passo 4 - Decidam LoRA ou full fine-tuning, com números

A entrega final: implementem em JavaScript a conversão de dataset e a validação de hiperparâmetros antes de qualquer treino, reaproveitando `local-lora-training-tool.js` como referência, e documentem a decisão final, comparando o ganho de qualidade de um full fine-tuning, se tiverem hardware pra rodar, ou no mínimo uma estimativa justificada do ganho esperado, contra o custo extra em memória e armazenamento, usando um limiar de decisão explícito como o do `full-vs-lora-tradeoff-tool.js`.

Entrega

- Caso real do próprio trabalho, aprovado pelo framework do Módulo 1.3, com volume pequeno documentado (Passo 1)
- NPV comparando custo fixo de GPU alugada versus custo local, usando os parâmetros do Módulo 1.3 (Passo 1)
- Pelo menos dois treinos LoRA locais, reais ou simulados e documentados, com tabela de trade-off: posto, parâmetros, val loss, memória, checkpoint (Passo 2)
- Exemplo de teste construído de propósito pra ser difícil, com resultado documentado (Passo 3)
- Código funcional em JavaScript, com testes automatizados, cobrindo conversão de dataset e validação de hiperparâmetros (Passo 4)
- Decisão final documentada: LoRA ou full fine-tuning, com critério explícito, não intuição (Passo 4)

Dica de execução

Rodar full fine-tuning de verdade pode não caber no hardware de vocês, mesmo pras últimas camadas: nesta disciplina, coube num Apple M5 Pro com vinte e quatro gigabytes, mas modelo maior ou máquina menor pode não caber. Se não for possível, documentem essa limitação com clareza e usem os números reais desta disciplina, Módulo 4.4, como referência de comparação, explicando por que esperam um resultado parecido ou diferente pro caso de vocês. E vale repetir o que já vale desde o Módulo 1.1: concluir que o caso de vocês não precisa de LoRA, ou nem de fine-tuning nenhum, também é uma entrega válida.